

釣行AIアプリ 企画・設計アーカイブ

これはアーカイブ資料です。本書は企画～MVP設計段階（2026年7月前半～中盤）に書かれた 企画書・提案書・要件定義書・WBS・コスト検討資料・AWS基本設計書・DynamoDBテーブル設計書の7つの資料を1本に統合したもの。「なぜその機能・技術を選んだか」という**当時の判断理由**を記録として残す目的で保管しており、**現在の正確な仕様ではない箇所が多く含まれる**（Cognito認証・釣具店検索・投稿編集・コスト保護レート制限・Claude Haiku移行など、この後の追加機能はほぼ反映されていない）。

現状の正しい情報は以下を参照:

- ・機能の全体像・使い方 → アプリまるわかりガイド.html
- ・Lambda関数単位の仕様 → 詳細設計.html
- ・ファイル構成・依存関係 → ファイル構成・依存関係.html
- ・AWSサービス構成 → AWS構成・サービス連携.html
- ・デプロイ手順 → README.md

1. 背景・課題（企画書より）

現状課題

- ・釣り場情報が複数サイトに分散している
- ・天気・潮汐・釣果情報を個別に調べる必要がある
- ・意思決定に時間がかかる（どこに行くか決まらない）
- ・経験者依存で初心者には判断が難しい

本質課題：「釣りに行く場所と釣り方の意思決定コストが高い」

→ AIにより「行くべき釣り場」と「釣り方」を即時提示することで解決する

想定ユーザー

- ・個人釣り愛好家（初心者～中級者）
- ・少人数グループ（友人・仲間）

差別化ポイント

競合（釣りスポット検索サイト・釣果投稿SNS・天気/潮汐アプリ）は「情報を探す」ことに留まるが、本アプリは統合分析・TOP3推薦・理由提示・即ナビ連携により「意思決定する」ところまで踏み込む。

2. 当初の機能要件（提案書・要件定義より統合）

機能	当初の想定内容
① TOP3おすすめ表示	現在地ベースで釣果期待値・天気・潮汐・距離・交通費を統合評価してランキング表示
② 地図表示	Google Mapsでピン表示・現在地からの距離・ナビ起動ボタン (https://www.google.com/maps/dir/?api=1&destination=lat,lng)
③ 釣果情報・魚種表示	その場所で釣れる魚種、過去投稿の釣果データ
④ AI理由表示	「なぜその釣り場が良いか」を自然言語で説明（例:「潮位が上げ始めでシーバスの活性が高い」）
⑤ お気に入り機能	スポット保存・メモ・ユーザー単位管理

当初のユーザーフロー

アプリ起動 → 現在地取得 → スポット取得 → AIスコアリング → TOP3表示（理由付き） → 詳細 or ナビ

当初のAPI要件

- GET /recommendations
- GET /spots
- GET /posts
- POST /favorites
- GET /favorites

実装ではこれらに加え、投稿編集・お気に入り削除・スポット写真設定・画像アップロード署名発行・AIチャット4種・プロフィール2種など、当初案には無かったAPIが多数追加されている（一覧は詳細設計.html参照）。

当初の非機能要件・制約

- レスポンス：1～2秒以内、可用性：99.9%（サーバレス前提）
- リアルタイム処理は禁止（コスト・遅延防止）。データはバッチ収集を基本とする
- AIはキャッシュ前提で利用
- セキュリティ：Google認証（Cognito）※要件定義段階では未実装、後日実装済み

3. スコアロジック（当初案）

```
score =
  fish_probability * 0.4
+ weather_score * 0.2
+ tide_score * 0.2
- distance_penalty * 0.1
- cost_penalty * 0.1
```

この計算式自体は現在もcalc_score() (generate_score.py) として実装に採用されている（重み係数の合計が0.8のため、満点条件でも上限は80点になる仕様。詳細設計.html参照）。

4. 当初のデータ設計（要件定義・DEテーブル設計・基本設計書より統合）

テーブル	当初の想定キー・項目
Spots	spotId(PK) / name / lat / lng / prefecture / type / createdAt。GSI1として prefecture(PK)+spotId(SK) を検討していたが未実装
Recommendations	spotId(PK) / score / fishTypes / reason / distance / cost / weatherScore / tideScore / updatedAt（最新1件のみ保持する上書き型）
Favorites	userId(PK) / spotId(SK) / memo / createdAt
Posts	当初案は spotId(PK)+createdAt(SK) の複合キーだったが、実装では postId 単一PKに変更（全件スキャン+アプリ側フィルタで個人利用規模なら問題ない判断）
Chats【2026-07-25設計時点で追加】	userId(PK)+chatId(SK)。1会話=1アイテムでmessages配列をまるごと保持

Users	設計時点では未実装（userIdは全テーブルで固定値"user-001"運用）。 googleId/email/name/createdAtを想定
-------	--

現在は上記に加え **UsageTable**（1日あたりのAI/API呼び出し回数記録、レート制限用）が存在し、Users テーブルも実装済み（Cognito認証導入に伴い、userIdは実際のGoogleアカウント単位のsubになっている）。最新のテーブル一覧はAWS構成・サービス連携.html「3. Amazon DynamoDB」参照。

5. 当初のAWSアーキテクチャ（基本設計書より）



リアルタイム処理は禁止し、バッチ中心で設計する方針（コスト削減・レスポンス安定化・外部依存排除）。バッチ処理の流れ：EventBridge → Lambda（データ収集）→ 天気・潮汐 API → DynamoDB保存 → AIスコア生成 → Recommendations更新。

当初のLambda構成案

バッチLambdaは当初「天気取得・潮汐取得・AIスコア生成・推薦更新」の4関数分割を想定していたが、実装では generateSpotScoreBatch 1関数に統合し、後に新規スポット探索を行う discoverSpotsBatch を追加する形になった。API Lambdaも当初は getRecommendations/getSpots/getPosts/createFavoriteの4本程度の想定だったが、実装では 投稿編集・チャット・プロフィール・釣具店検索等を含む17本まで拡張している（最新の関数一覧は詳細設計.html参照）。

6. AI設計方針の変遷

設計当初の方針: スコア自体はAIに任せずルールベース計算式で決定論的に算出し、自然言語の「理由 (reason)」生成にのみAIを使う、という役割分担は設計当初から一貫している。当初案ではGPT-4.1/Claude併用も検討したが、実装では理由生成のみGeminiを採用し、モデル廃止・世代交代に追従するため固定バージョン名ではなく自動追従エイリアス (gemini-flash-latest) を使う設計にしていた。

2026-07-26追記: このGemini採用は、スポット数が100件を超えた段階で「1日20リクエストまで」という無料枠の壁を1回の「AI分析を実行」で使い切ってしまう 障害を引き起こした。この日、理由生成・AI相談チャット・新スポットの魚種推測の3箇所すべてをClaude Haiku (Anthropic API) に移行し、無料枠の回数制限という 構造的リスク自体を解消した。経緯の詳細は詳細設計.htmlの該当セクション参照。

7. コスト前提・技術選定理由（検討資料より）

コスト前提（設計時点）：個人開発・ユーザー数1〜2人・低頻度アクセス（1日数十リクエスト）・リアルタイム処理なし。目標コスト：月0〜1,500円以内。

採用技術と理由

技術	採用理由
AWS Lambda	サーバ不要で固定費ゼロ、バッチ処理と相性が良い
DynamoDB	スケーラブルで運用不要（オンデマンド課金）
CloudFront	高速配信、S3単体よりUX改善
AI API（外部LLM）	釣行判断ロジックの理由生成のみに限定利用。リアルタイム呼び出しは行わずコスト抑制

非採用技術と理由

技術	非採用理由
EC2	常時課金が発生（最小でも数千円〜）
RDS	インスタンス固定費（最低1,000円〜）
リアルタイムスクレイピング	外部依存で遅く、APIコストも増加

コスト設計思想：「単価削減」ではなく「発生回数削減」を本質とする（リアルタイムを捨てる／バッチ処理で回数を減らす／キャッシュ前提設計／AI呼び出しを最小化）。この思想は2026-07-26のreason使い回しキャッシュ実装にもそのまま引き継がれている。

コスト見積もりの実額は設計時点（Gemini・Lambda関数半分以下の規模）のもので、現在の 実際のサービス構成・料金体系（Claude API従量課金、Cognito、追加Lambda多数）とは乖離している。最新のコスト構造はAWS構成・サービス連携.html「5. なぜこの構成にしたか」 およびREADME.md「10. 利用料アラート」を参照。

8. 開発フェーズ・工数見積もり（WBSより）

Phase 1：企画・設計

Phase 2：AWS基盤構築（S3・CloudFront・API Gateway・Lambda・DynamoDB・Cognito）

Phase 3：バックエンド実装（API・スコアリングロジック）

Phase 4：フロントエンド実装（PWA・TOP3画面・地図・詳細画面・お気に入りUI）

Phase 5：AIロジック実装（理由生成プロンプト・バッチ統合・キャッシュ設計）

Phase 6：統合テスト

Phase 7：デプロイ（GitHub Actions・S3自動デプロイ・Lambda自動更新）

工数目安（個人開発想定）：設計2～3日／AWS構築2～4日／バックエンド3～5日／フロント3～5日／AI実装2～3日／統合2日 → 合計2～3週間でMVP完成を想定。

リスク管理（当初想定）

- 外部API障害 → キャッシュで吸収
- AIコスト増 → バッチ化で制御（実際にGemini無料枠枯渇という形で顕在化し、reason使い回し＋Claude移行で対応）
- スコア精度不足 → ロジック調整

9. 将来拡張アイデア（未着手、提案書より）

- 仲間グループでのスポット共有
- 釣果予測AIの強化
- リアルタイム潮汐最適化
- 釣具レコメンド
- 収益化（プレミアム機能・広告モデル・釣具/釣り場アフィリエイト）

10. まとめ

本プロダクトは企画段階から一貫して「釣り情報の検索アプリ」ではなく「釣行意思決定を支援するAIアプリ」として設計されてきた。MVP以降、認証・投稿編集・AI相談チャット・釣具店検索・コスト保護など多数の機能が追加されているが、「実在データ（Places API・Open-Meteo）とAIによる説明生成を分離し、AIには存在しない場所を作らせない」「リアルタイム処理を避けバッチ中心でコストを抑える」という設計思想の根幹は企画当初から変わっていない。